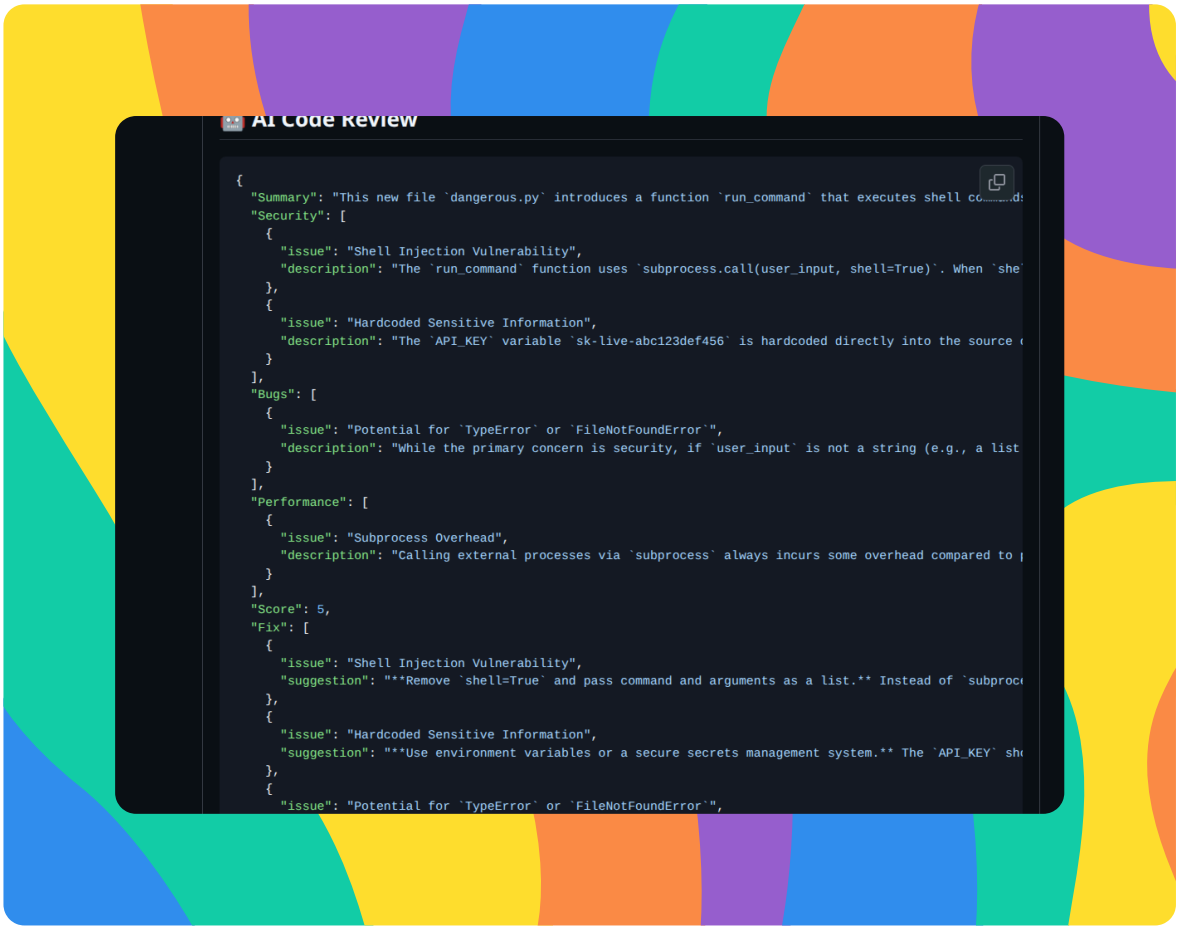
Review GitHub Pull Requests with Gemini

Ahmed Tetteh



Ahmed Tetteh
NextWork Student

Introducing Today's Project!

In this project, I'm going to build an AI code reviewer system with Gemini. I'm doing this project because many pull resquests present as bottlenecks, especially when waiting for merges or reviews - adding an AI model at this step can significant reduce the errors and speed up pull requests.

Key tools and concepts

The key tools I used include GitHub Actions, Python and Gemini API. Key concepts I learnt include how to build a Python script that calls the Gemini API to analyze code diffs,creating a GitHub Actions workflow triggered on every pull request, posted automated review comments using github-script and auto-labeled PRs by AI review severity.

Challenges and wins

This project took me approximately 4 hrs (Including code review, troubleshooting and background studies). The most challenging part was creating the pull request workflow after perfoming my local test.

Ahmed Tetteh
NextWork Student

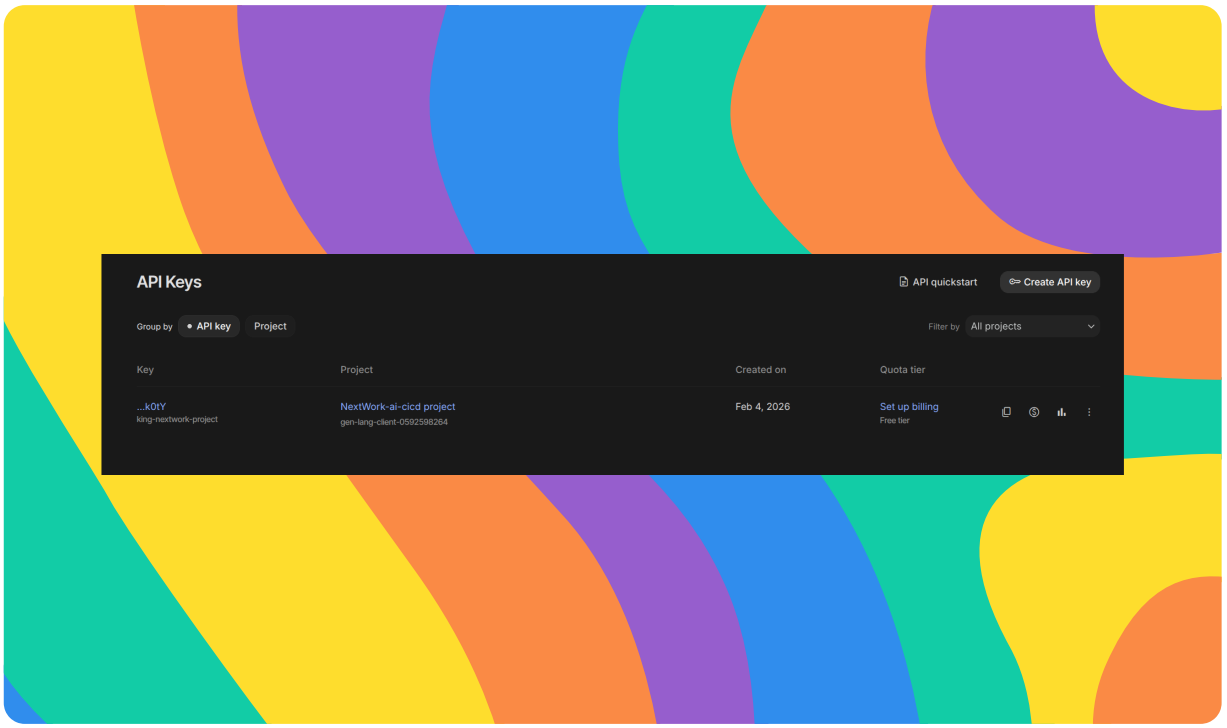
Why I did this project

I did this project today to learn how to about how AI agents can be integrated into the GitHub actions workflow.

Ahmed Tetteh
NextWork Student

Getting the Gemini API Key

In this step, I'm going to get access to the Gemini API key from Google AI studio. The Gemini API is a tool that allows me to integrate Gemini into my workflow. I need an API key because it authenticates my requests to Gemini.



Securing the API key

I got my API key from Google AI studio. API keys need to be kept secure because it represents your password for authenticating your requests to Gemini. Every request made if counted against your daily quota of 1500 request/day, which when stolen by someone else or abused - can incur charges or potentially block your access to Gemini.

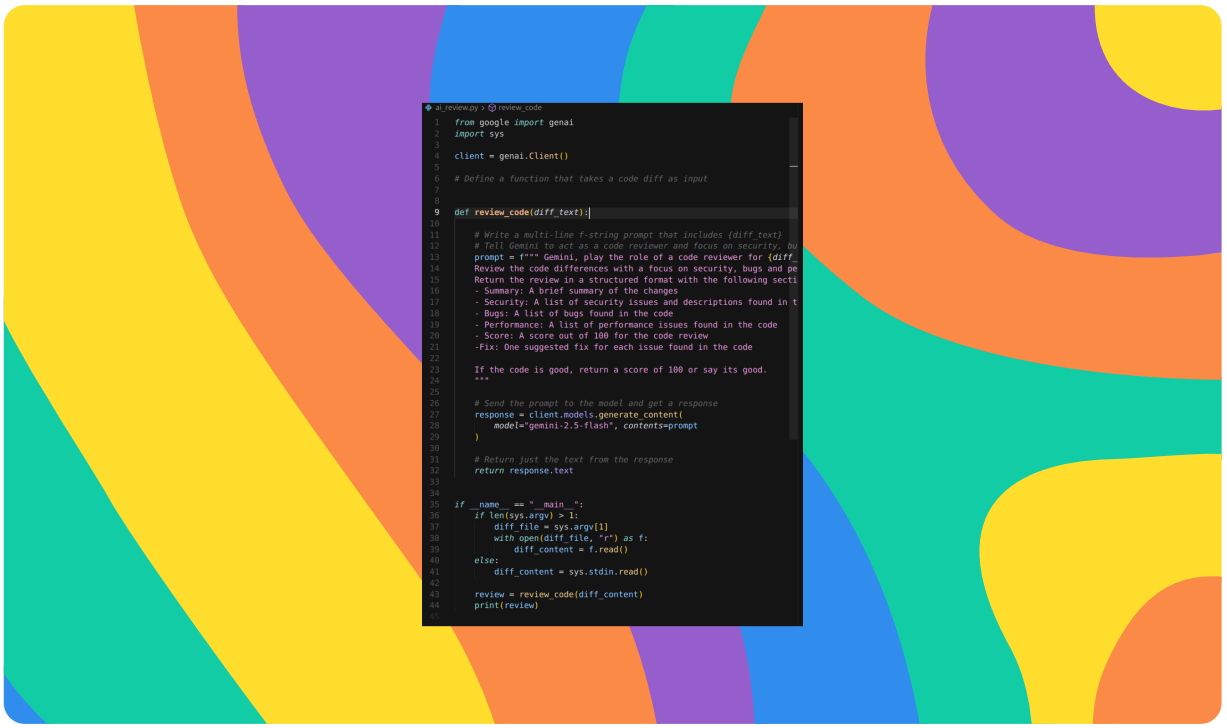
Installing the dependency

I installed google-genai by running "pip install -r requirements.txt" .This package lets my code interact with Googl's AI model (Gemini).

Ahmed Tetteh
NextWork Student

Building the AI Review Script

In this step, I'm going to build a Python script that analyzes my code for changes within it. A code diff is any tool's output that clearly shows the differences between two files, sets of data or modified code.



How the review function works

The review_code function takes a diff_text parameter which refers to the difference in code. The f-string prompt tells Gemini to act a code reviewer and properly analyze the code diff, identlfy the errors in the changes and provide fixes. The response.text property returns only the texts in the AI generated responses.

The complete script

The complete script includes importing the necessary libraries, the sending the prompt along the the code diff as an input to Gemini 2.5 model, generating a response using the model and printing the response on the terminal. The main block allows the script to be run from the terminal only when the script is executed directly, not as a sub module.. This lets me pass diff files using either the standard input or code difference.

Ahmed Tetteh
NextWork Student

Testing the Script Locally

In this step, I'm going to test my script by creating a sample diff file. A sample diff is a file that contains a saummary of changes between two versions of a file or set of files. When I run my test, I expect to see a review comment from Gemini, and some suggestions for fixes.

Identifying the security vulnerability

The sample diff contains a SQL injection vulnerability which can negativley impact your app or resource, such as a database. An attacker could exploit this by feeding the malicious code into your form fields of a database, and effectively gaining access to the database.

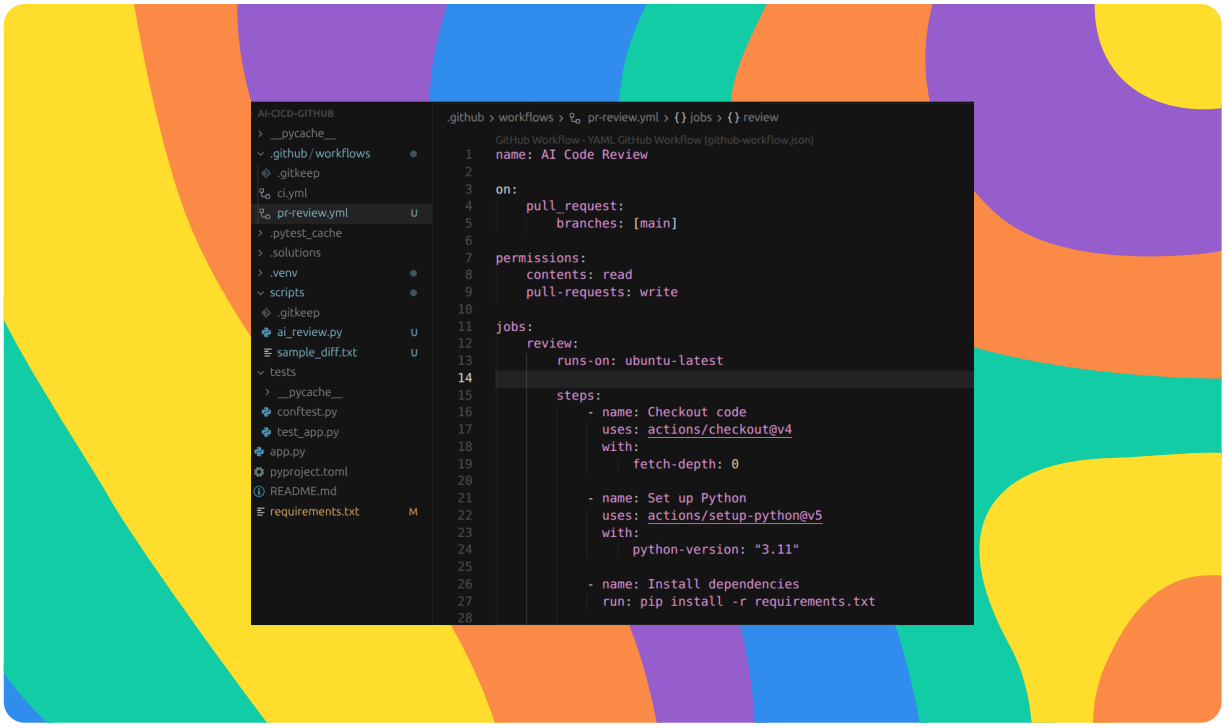
Gemini's review results

When I ran the script, Gemini detected the SQL injection vulnerability. The review mentioned a "get_user" function used to retrieve user information by directly embeding the username parameter, and when this function called many times, it can lead to database connection resource leakage and consume system resources. This result means my python script works well by passing the diff_text file as an argument to the function "review_code" in the ai_review.py script, then sends the prompt and the diff_text file to Gemini, and finally returns or prints out the response to the terminal.

Ahmed Tetteh
NextWork Student

Configuring the GitHub Actions Workflow

In this step, I'm going to add my API Key to GitHub secrets, create a GitHub actions workflow and push my changes to GitHub. I used GitHub secrets to store by API key because it widly insecure to keep a secret key or credential in a public repository or an open space.



Storing the API key as a GitHub Secret

A GitHub Secret is an encryted environment variable stored in your GitHub repository settings and used for sensitive data. I need one because my Gemini API key is a secret key used for making requests on my behalf to Gemini, which counts against my quota.

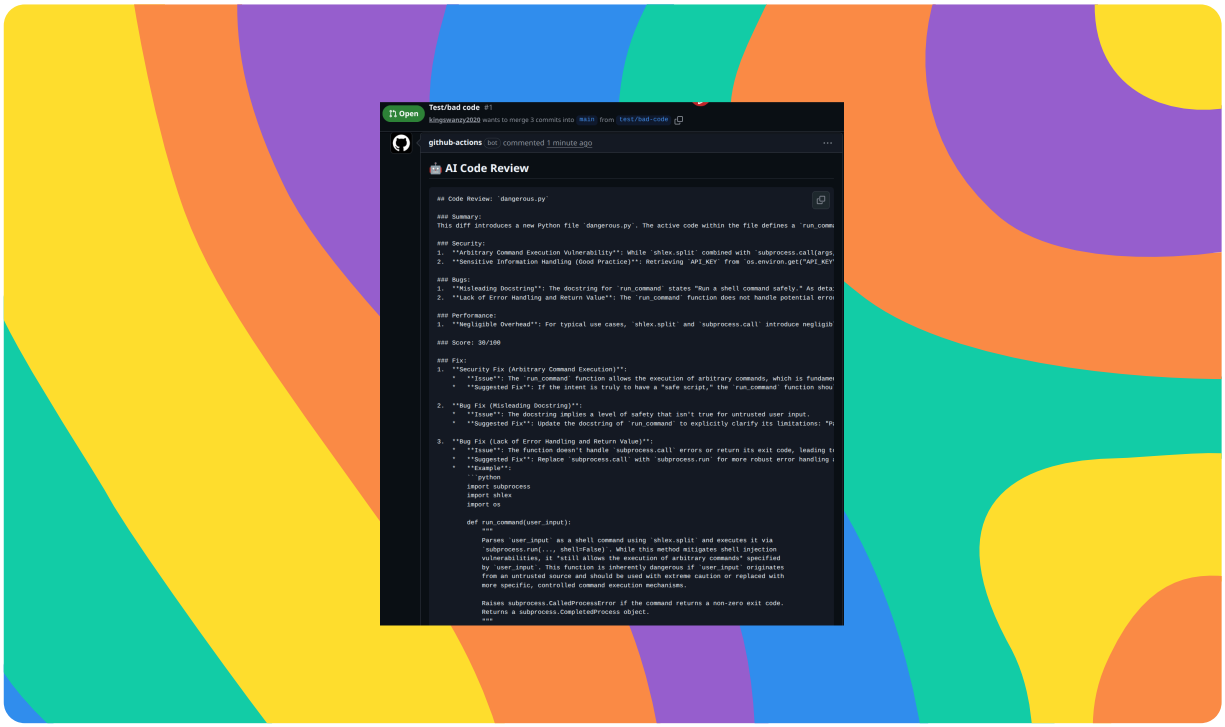
How the workflow triggers

This workflow creates an AI review with PR comments. It triggers when a pull request is opened or updated on the main branch, then it goes through a series of steps to finally ouput the comments from Gemini.

Ahmed Tetteh
NextWork Student

Running the AI Code Review on a Pull Request

In this step, I'm going to test my PR review workflow.



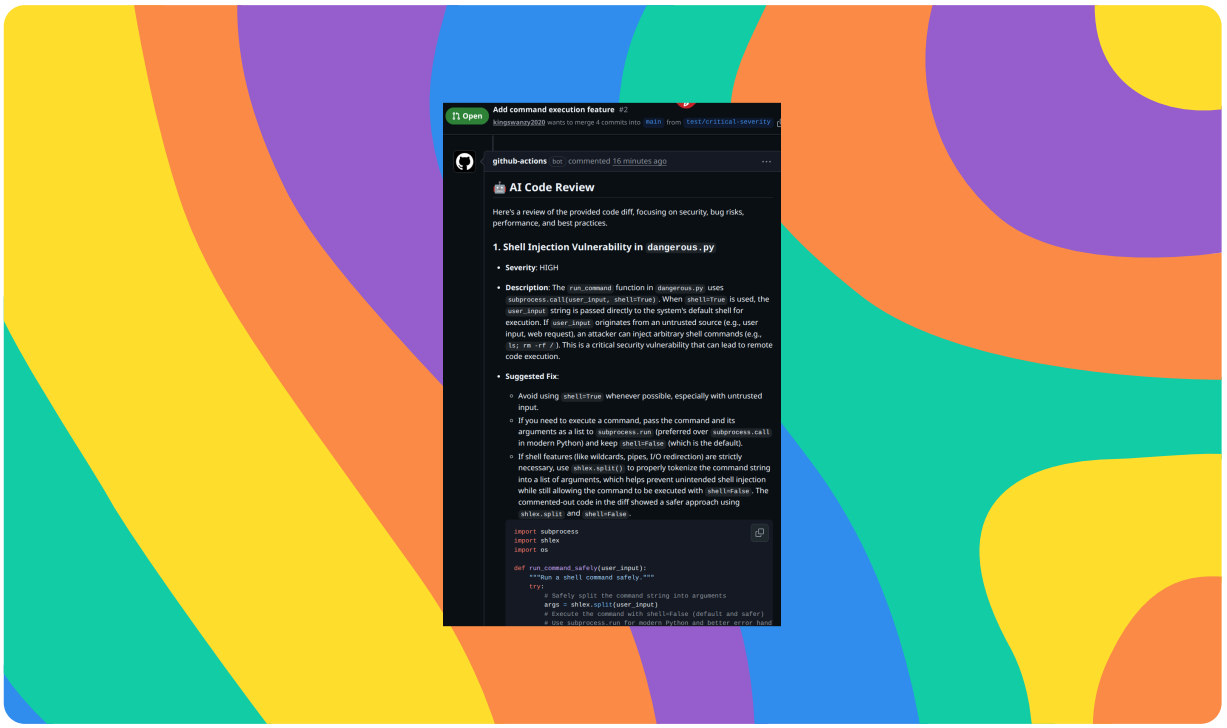
Security issues detected

The AI reviewer found a shell injection vulnerability in my script. My script ran through the following steps.: - Checkout code: Fetches your repository's code. - Set up Python: Installs the required Python version. - Install dependencies: Installs Python packages from your requirements.txt file. - Get PR diff: Generates the code differences from the pull request. - Run AI review: Executes your ai_review.py script with the code diff and captures the AI's review. - Post review comment: Posts the AI's review as a comment on the pull request. These are security risks because hardcoded API key poses a security risk for resource access and there is also a potential for `TypeError` or `FileNotFoundError if user_input is not a string.

Ahmed Tetteh
NextWork Student

Auto-Labeling PRs by Severity

In this project extension, I'm going to include the ability to auto-label PRs by severity based on the review. I'm doing this secret mission because It makes it very easy to automically identify the severity of the vulnerability in our workflow.



Modifying the AI review script

I updated the prompt to request a summary of the severity. The SEVERITY_SUMMARY line format is a machine readable format created to parse strings in the workflow.

Ahmed Tetteh
NextWork Student

How labels help triage PRs

The three labels I created are Critical, Warning and Good. The workflow applied the severity High, which corresponds to the Critical label because of the structured machine format for the severity cases in each of the AI generated responses. This automation helps reviewers by quickly spotting a severity level of their workflow, and attending to it immediatley based on the level

The place to learn & showcase your skills

Check out nextwork.org for more projects